Lesson 01 Demo 02

Creating HTML Elements for JSON Server

Objective: To create an HTML file that dynamically retrieves and displays user data from a JSON server using JavaScript, showcasing the integration of HTML, CSS, and JavaScript for interacting with a REST API

Tools required: Kali Linux and Visual Studio Code

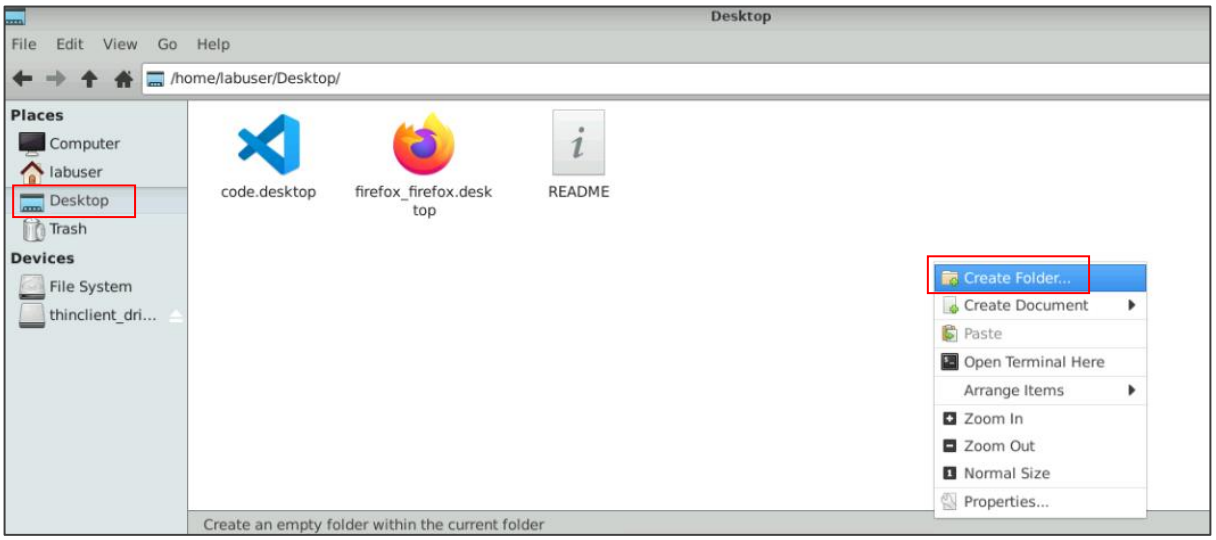
Prerequisites: Refer to Lesson_01_Demo_01_Creating_and_Using_a _JSON_Server_File

Steps to be followed:

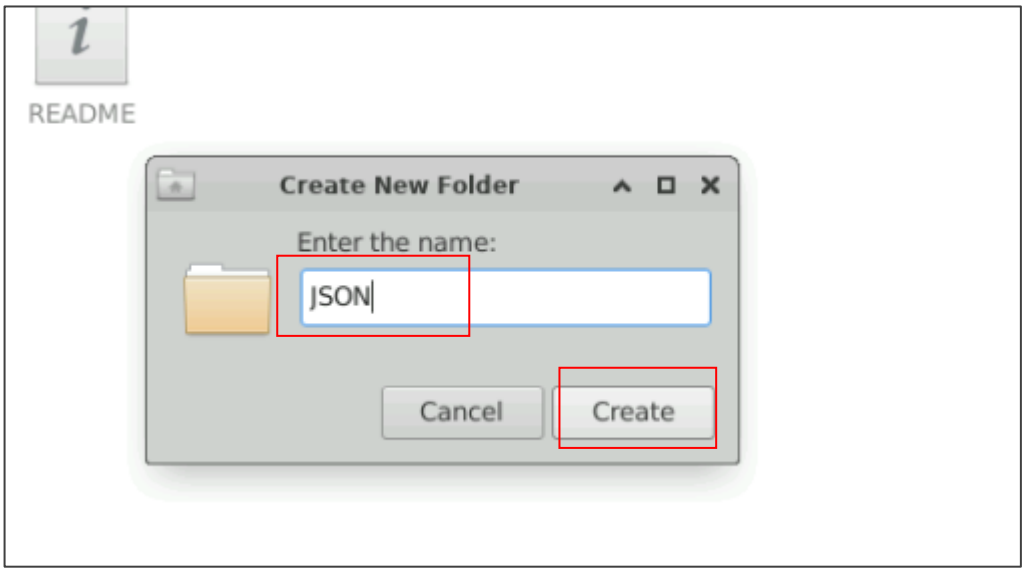
1. Set up the HTML file
2. Integrate Java script for data fetching
3. Run and view the application

Step 1: Set up the HTML file

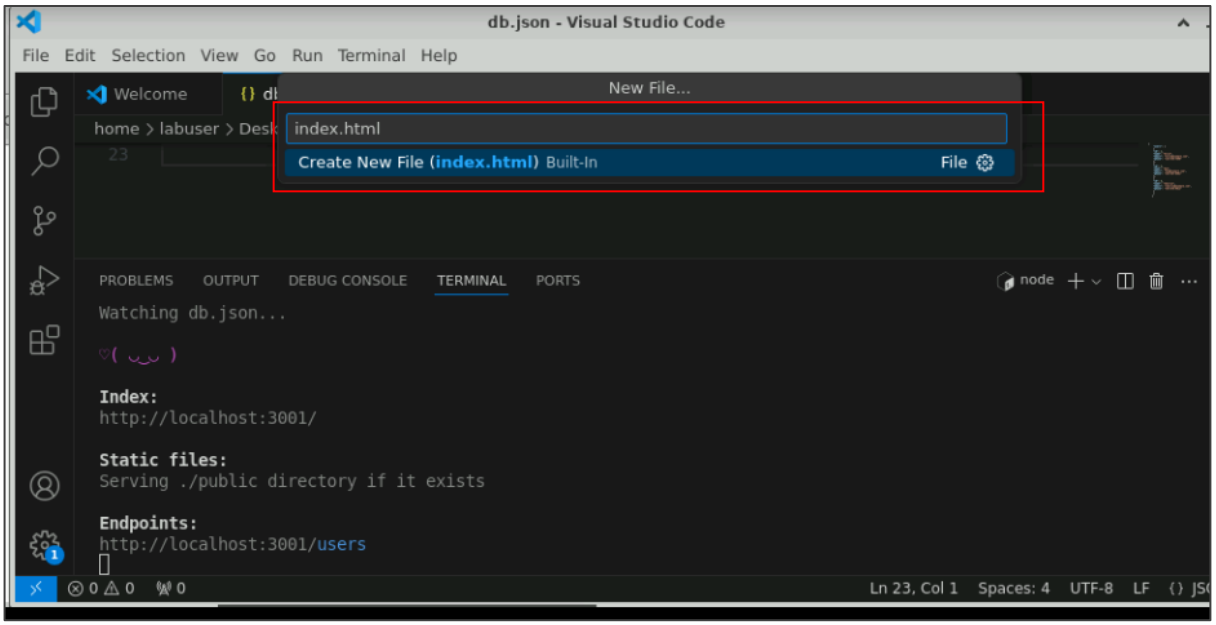
1.1 Right-click on an empty space on your **Desktop** and select **Create Folder**



1.2 Name the folder **JSON** and click **Create** to confirm



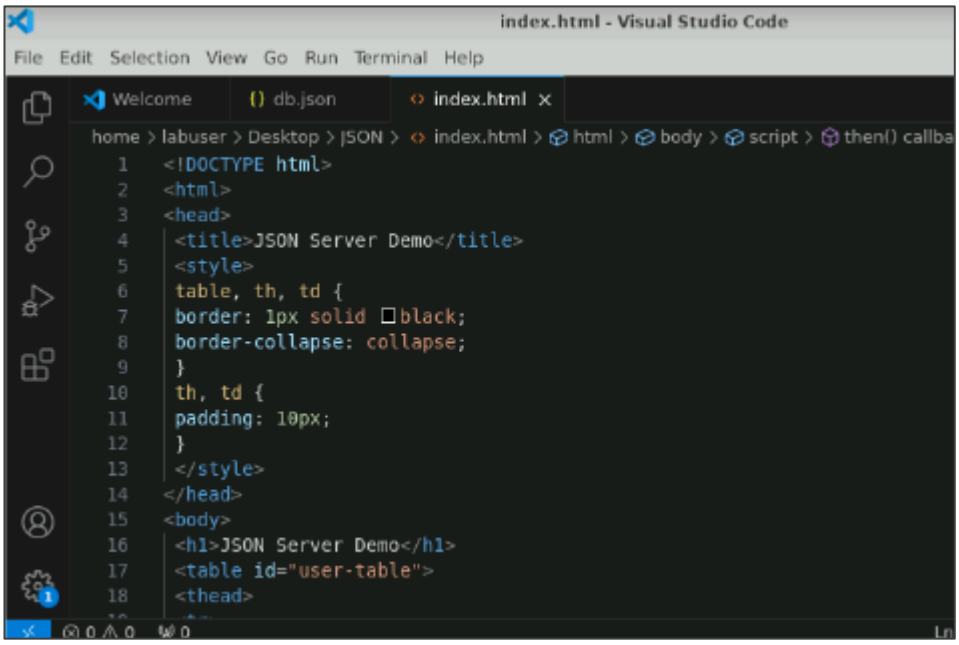
1.3 Open Visual Studio Code and create a new file named **index.html** inside the JSON folder



1.4 Copy and paste the following HTML and CSS code into the index.html file:

```
<!DOCTYPE html>
<html>
 <head>
  <title>JSON Server Demo</title>
   <style>
```

```
      table, th, td {
        border: 1px solid black;
        border-collapse: collapse;
      }
      th, td {
        padding: 10px;
      }
    </style>
  </head>
  <body>
    <h1>JSON Server Demo</h1>
    <table id="user-table">
      <thead>
        <tr>
          <th>ID</th>
          <th>Name</th>
         <th>Email</th>
          <th>Phone</th>
        </tr>
      </thead>
      <tbody>
        <!-- The table body will be populated with the data from the JSON server
  -->
      </tbody>
    </table>
    <script>
    </script>
  </body>
</html>
```



Step 2: Integrate Java script for data fetching

2.1 Add the following Java script inside the <script> Tag to fetch and display user data from the JSON server:

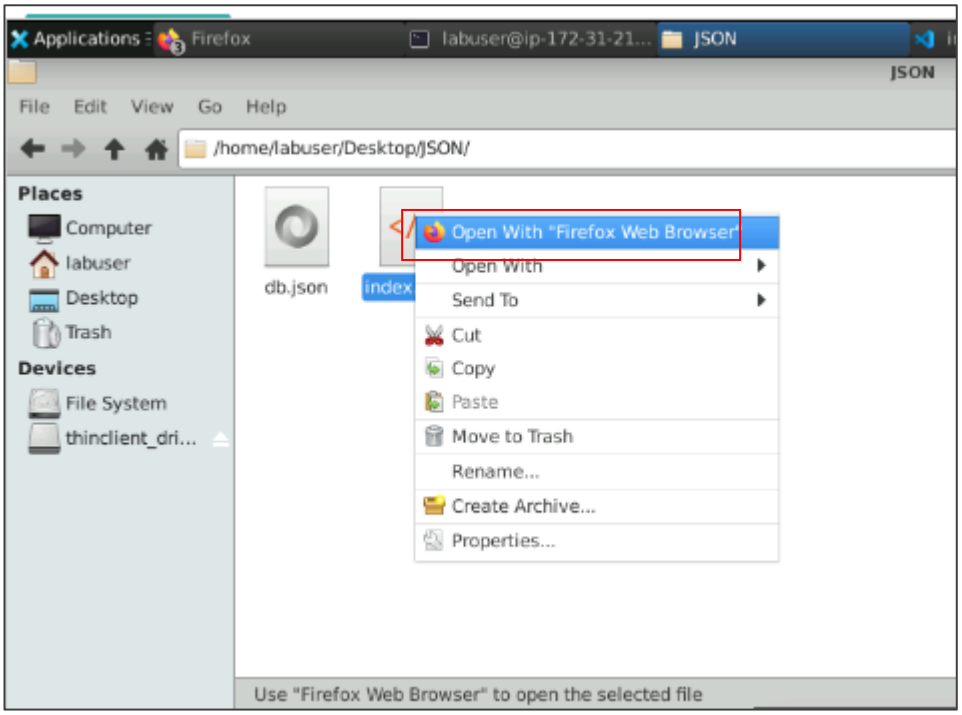
```
// Use the fetch API to get the data from the JSON server
 fetch('http://localhost:3000/users')
    .then(response => response.json()) // Convert the response to JSON
    .then(data => {
       // Get the table body element
       let tbody = document.getElementById('user-
 table').getElementsByTagName('tbody')[0];
       // Loop through the data array
       for (let user of data) {
          // Create a new table row element
          let tr = document.createElement('tr');

          // Create table cell elements for each user property
          let td_id = document.createElement('td');
           let td_name = document.createElement('td');
          let td_email = document.createElement('td');
          let td_phone = document.createElement('td');

          // Set the text content of each table cell
          td_id.textContent = user.id;
          td_name.textContent = user.name;
          td_email.textContent = user.email;
          td_phone.textContent = user.phone;

          // Append the table cells to the table row
          tr.appendChild(td_id);
          tr.appendChild(td_name);
          tr.appendChild(td_email);
           tr.appendChild(td_phone);

          // Append the table row to the table body
          tbody.appendChild(tr);
       }
    })
       .catch(error => {
       console.error(error); // Handle any errors});
```

```
30  <script>
31      // Use the fetch API to get the data from the JSON server
32      fetch('http://localhost:3001/users')
33      .then(response => response.json()) // Convert the response to JSON
34      .then(data => {
35      // Get the table body element
36      let tbody = document.getElementById('user-table').getElementsByTagName('tbody')[0];
37      // Loop through the data array
38      for (let user of data) {
39      // Create a new table row element
40      let tr = document.createElement('tr');
41      // Create four table cell elements for each user property
42      let td_id = document.createElement('td');
```

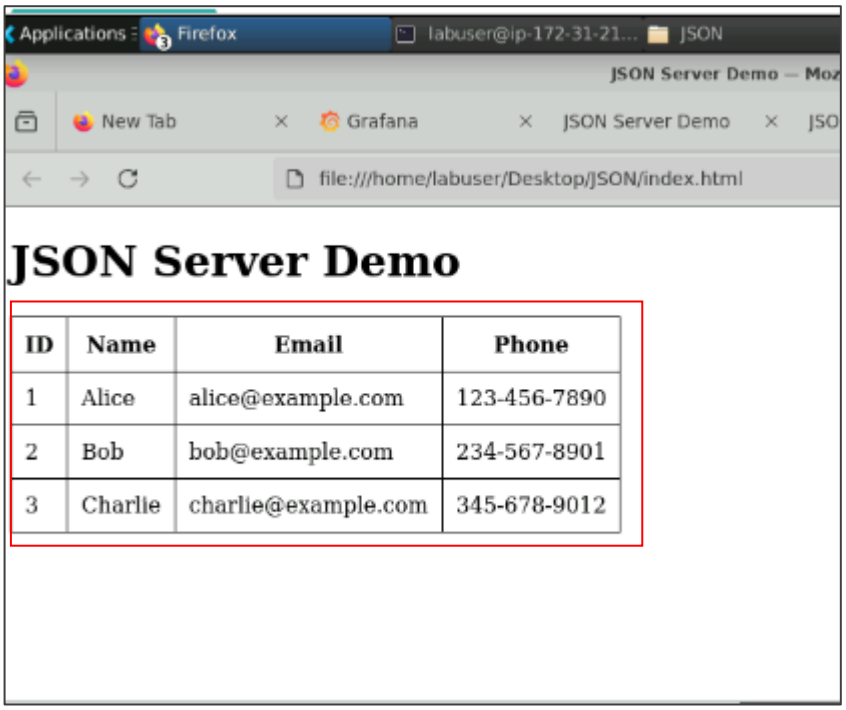
1 ⚠ 0 0 Ln 36, Col 76 Spaces: 3 UTF-8 LF HTML

Step 3: Run and view the application

3.1 Save the index.html file, right-click on the index.html file, and click on **Open With "Firefox in a Web Browser"**



3.2 View the table with user data



The webpage should display a table populated with user data fetched from the JSON server.

By following these steps, you have successfully created an HTML file that fetches and displays user data from a JSON server in a table format. This exercise demonstrates how to integrate HTML, CSS, and JavaScript to interact with a REST API dynamically.